CENTRAL BOARD OF SECONDARY EDUCATION

Senior Secondary School Examination - 2026

PRACTICAL FILE

SUB: COMPUTER SCIENCE (Code: 083)

SUBMITTED BY:	Kunwar Naitik Singh	**SUBMITTED TO:**	Ardhenddra Singh Sir
ROLL NUMBER:	21	**DESIGNATION:**	PGT Computer Science
CLASS / SEC:	XII - A	**SCHOOL:**	Maa Durga Ji Senior Sec. Vidhyalaya

CERTIFICATE OF EXCELLENCE

This is to certify that Master Kunwar Naitik Singh, a student of class **XII (Science)** has successfully completed the term laboratory practical file in **Computer Science (083)** prescribed by the **Central Board of Secondary Education (CBSE)** for the academic session **2026-2027**.

All the works and programs included in this project/file are completed by the candidate individually under proper guidance, supervision, and instruction of the internal faculty.

Internal Examiner	**External Examiner**

TABLE OF CONTENTS

S.No.	Topic / Program Statement	Remarks
Chapter 1: Review of Python Programming Basics & Functions		
1	Program to check if a string is palindrome or not using a user-defined function.	Verified
2	Program to compute the GCD/HCF of two numbers using mathematical recursion/ iteration.	Verified
3	Program using functions to count occurrences of elements in a list using a dictionary.	Verified
4	Program to pass a list to a function and modify its elements (Even elements multiplied by 2, Odd by 3).	Verified
5	Program to find the second largest element in a user-input list without built-in sorting.	Verified
Chapter 2: Text File Handling		
6	Program to read a text file line by line and print lines starting with a specific alphabet 'A'.	Verified
7	Program to count total vowels, consonants, uppercase, and lowercase characters in a text file.	Verified
8	Program to copy all lines from 'source.txt' to 'dest.txt' excluding lines containing character '#'.	Verified
9	Program to calculate and display the total number of words starting with 'the' or 'this' in a file.	Verified
10	Program to replace all spaces in a text file with a hyphen '-' and display the content.	Verified
Chapter 3: Binary File Handling		
11	Program to write and read student records (Roll, Name, Marks) using the pickle module.	Verified
12	Program to search for a student record based on Roll Number in a binary file and update marks.	Verified

S.No.	Topic / Program Statement	Remarks
13	Program to delete a specific item record from a binary file 'inventory.dat' based on ItemID.	Verified
14	Program to count records in an employee binary file where salary is greater than 50,000.	Verified
15	Program to append new records into an existing standard binary file safely.	Verified
Chapter 4: CSV File Handling		
16	Program to create a CSV file containing User ID, Username, and Password fields.	Verified
17	Program to read a CSV file and search for an entry based on user-specified Username.	Verified
18	Program to copy specific rows from a source CSV file to a target CSV based on a criterion.	Verified
19	Program to count the total rows/records in a CSV file excluding the header row.	Verified
20	Program to update the salary field of a given Employee ID inside a CSV file cleanly.	Verified
Chapter 5: Data Structures (Stacks)		
21	Implementation of a Stack structure in Python for a book repository (Push, Pop, Display).	Verified
22	Implementation of a Stack using lists to filter and hold only numbers divisible by 5.	Verified
23	Program to implement Peek and checking Empty states alongside regular Stack operations.	Verified
24	Stack implementation to manage customer profiles (IDs and names) in an linear array list.	Verified
25	Program to reverse a user-entered string using a character-based Stack structure.	Verified
Chapter 6: Database Management and Python-SQL Connectivity		
26	Program to connect Python with MySQL database and print a success/failure message.	Verified
27	Program to insert dynamic user-input records into a MySQL database table 'Student'.	Verified
28	Program to fetch and display all rows from a database table using the fetchall() cursor method.	Verified

S.No.	Topic / Program Statement	Remarks
29	Program to search and display specific records matching a parameterized condition using fetchone().	Verified
30	Program to delete a record from a database table dynamically and commit changes to the backend.	Verified

CHAPTER 1: REVIEW OF PYTHON BASICS & FUNCTIONS

Program 1: Palindrome Check Function

Write a user-defined function is_palindrome(s) that checks whether a given string is a palindrome and returns True or False. Implement a driver block to accept user input.

SOURCE CODE:

```
def is_palindrome(s):
    # Normalize string by removing spaces and making it lowercase
    clean_s = "".join(s.split()).lower()
    return clean_s == clean_s[::-1]

# Driver code
user_str = input("Enter a string: ")
if is_palindrome(user_str):
    print(f"'{user_str}' is a palindrome.")
else:
    print(f"'{user_str}' is not a palindrome.")
```

OUTPUT:

```
Enter a string: Racecar
'Racecar' is a palindrome.

Enter a string: Computer Science
'Computer Science' is not a palindrome.
```

Program 2: Greatest Common Divisor (GCD) Calculation

Write a function compute_gcd(a, b) to calculate the Greatest Common Divisor of two integers using the Euclidean algorithm loop structure.

SOURCE CODE:

```
def compute_gcd(a, b):
    while b:
        a, b = b, a % b
    return a

# Driver code
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
gcd = compute_gcd(num1, num2)
print(f"The GCD of {num1} and {num2} is: {gcd}")
```

OUTPUT:

```
Enter first number: 60
Enter second number: 48
The GCD of 60 and 48 is: 12
```

Program 3: Element Frequency Counter using Dictionary

Write a Python program containing a function count_frequency(lst) that computes the operational counts of every distinct element in a list and returns a populated dictionary.

SOURCE CODE:

```
def count_frequency(lst):
    freq_dict = {}
    for item in lst:
        freq_dict[item] = freq_dict.get(item, 0) + 1
    return freq_dict

# Driver code
sample_list = [1, 2, 3, 1, 2, 1, 4, 5, 2, 4]
print("Original List:", sample_list)
result = count_frequency(sample_list)
print("Frequency Dictionary:")
for key, val in result.items():
    print(f"Element {key} occurs {val} times")
```

OUTPUT:

```
Original List: [1, 2, 3, 1, 2, 1, 4, 5, 2, 4]
Frequency Dictionary:
Element 1 occurs 3 times
Element 2 occurs 3 times
Element 3 occurs 1 times
Element 4 occurs 2 times
Element 5 occurs 1 times
```

Program 4: List Manipulation Function

Write a function modify_list(lst) that accepts a numerical list and multiplies all even-indexed elements by 2 and odd-indexed elements by 3. Print the modified list.

SOURCE CODE:

```
def modify_list(lst):
    for i in range(len(lst)):
        if i % 2 == 0:
            lst[i] *= 2
        else:
            lst[i] *= 3

# Driver code
numbers = [10, 20, 30, 40, 50]
print("Before modification:", numbers)
modify_list(numbers)
print("After modification: ", numbers)
```

OUTPUT:

```
Before modification: [10, 20, 30, 40, 50]
After modification:  [20, 60, 60, 120, 100]
```

Program 5: Second Largest Number without Sorting

Write a function find_second_largest(numbers) that iterates through a list to identify the second largest value without invoking built-in sort functionalities.

SOURCE CODE:

```
def find_second_largest(numbers):
    if len(numbers) < 2:
        return None

    first = second = float('-inf')
    for num in numbers:
        if num > first:
            second = first
            first = num
        elif num > second and num != first:
            second = num

    return second if second != float('-inf') else None

# Driver code
my_list = [12, 35, 1, 10, 34, 35, 20]
print("List:", my_list)
ans = find_second_largest(my_list)
print("Second largest element is:", ans)
```

OUTPUT:

```
List: [12, 35, 1, 10, 34, 35, 20]
Second largest element is: 34
```

CHAPTER 2: TEXT FILE HANDLING

Program 6: Display Lines Starting with 'A'

Write a function display_lines_A() to read a text file named "story.txt" and display only those lines that begin with the character 'A' (case-insensitive).

SOURCE CODE:

```
def display_lines_A():
    # Creating a sample file for demonstration
    with open("story.txt", "w") as f:
        f.write("Apple is good for health.
Orange is a citrus fruit.
All roads lead to Rome.
Keep learning.
")

    # Reading and filtering lines
    with open("story.txt", "r") as file:
        lines = file.readlines()
        print("Lines starting with 'A':")
        for line in lines:
            if line.upper().startswith('A'):
                print(line.strip())

# Driver code
display_lines_A()
```

OUTPUT:

```
Lines starting with 'A':
Apple is good for health.
All roads lead to Rome.
```

Program 7: Count Characters, Vowels and Consonants

Write a function count_file_stats() that reads a file "notes.txt" and explicitly counts vowels, consonants, uppercase characters, and lowercase characters.

SOURCE CODE:

```
def count_file_stats():
    # Setup sample file
    with open("notes.txt", "w") as f:
        f.write("Welcome to CBSE Class 12 Computer Science!")

    vowels = "AEIOUaeiou"
    v_count = c_count = up_count = low_count = 0

    with open("notes.txt", "r") as file:
        content = file.read()
        for ch in content:
            if ch.isalpha():
                if ch.isupper():
                    up_count += 1
                if ch.islower():
                    low_count += 1
                if ch in vowels:
                    v_count += 1
                else:
                    c_count += 1

    print(f"Vowels: {v_count}")
    print(f"Consonants: {c_count}")
    print(f"Uppercase Characters: {up_count}")
    print(f"Lowercase Characters: {low_count}")

# Driver code
count_file_stats()
```

OUTPUT:

```
Vowels: 13
Consonants: 22
Uppercase Characters: 6
Lowercase Characters: 29
```

Program 8: Copy File Excluding Comments

Write a function filter_copy() to copy all lines from a file "source.txt" to a new file "dest.txt", excluding lines that begin with a comment character '#'.

SOURCE CODE:

```
def filter_copy():
    # Setup base content
    with open("source.txt", "w") as sf:
        sf.write("# Main Configuration
port = 8080
# Debug Mode
debug = True
")

    with open("source.txt", "r") as src, open("dest.txt", "w") as dest:
        for line in src:
            if not line.strip().startswith("#"):
                dest.write(line)

    # Display output destination contents
    print("Contents of dest.txt:")
    with open("dest.txt", "r") as df:
        print(df.read())

# Driver code
filter_copy()
```

OUTPUT:

```
Contents of dest.txt:
port = 8080
debug = True
```

Program 9: Count Word Specific Frequency ('the' / 'this')

Write a function count_words_the_this() to parse a text file "article.txt" and return the specific cumulative frequency count of the individual words "the" and "this".

SOURCE CODE:

```
def count_words_the_this():
    with open("article.txt", "w") as f:
        f.write("The quick brown fox jumps over the lazy dog. This is an absolute
truth.")

    count = 0
    with open("article.txt", "r") as file:
        for line in file:
            words = line.split()
            for w in words:
                clean_w = w.strip(".,!?").lower()
                if clean_w in ["the", "this"]:
                    count += 1

    print("Total occurrences of 'the' and 'this':", count)

# Driver code
count_words_the_this()
```

OUTPUT:

```
Total occurrences of 'the' and 'this': 3
```

Program 10: Space to Hyphen Formatter

Write a program to open a text file "data.txt", substitute every whitespace/space character with a hyphen symbol '-', and print the updated text layout.

SOURCE CODE:

```
def space_to_hyphen():
    with open("data.txt", "w") as f:
        f.write("Python Programming Language Environment")

    with open("data.txt", "r") as file:
        content = file.read()

    modified_content = content.replace(" ", "-")
    print("Modified Content Layout:")
    print(modified_content)

# Driver code
space_to_hyphen()
```

OUTPUT:

```
Modified Content Layout:
Python-Programming-Language-Environment
```

CHAPTER 3: BINARY FILE HANDLING

Program 11: Write and Read Student Records

Write a program to read student data structured as nested lists/dictionaries, use pickle.dump() to write into "student.dat", and read it back sequentially via pickle.load().

SOURCE CODE:

```
import pickle

def write_binary():
    records = [
        {"roll": 1, "name": "Amit Sharma", "marks": 92.5},
        {"roll": 2, "name": "Bhavna Singh", "marks": 95.0},
        {"roll": 3, "name": "Chirag Gupta", "marks": 88.0}
    ]
    with open("student.dat", "wb") as f:
        pickle.dump(records, f)
    print("Binary data successfully dumped.")

def read_binary():
    print("
Reading Records from student.dat:")
    with open("student.dat", "rb") as f:
        data = pickle.load(f)
        for rec in data:
            print(f"Roll: {rec['roll']}, Name: {rec['name']}, Marks: {rec['marks']}")

# Driver code
write_binary()
read_binary()
```

OUTPUT:

```
Binary data successfully dumped.

Reading Records from student.dat:
Roll: 1, Name: Amit Sharma, Marks: 92.5
Roll: 2, Name: Bhavna Singh, Marks: 95.0
Roll: 3, Name: Chirag Gupta, Marks: 88.0
```

Program 12: Search and Update Record Marks

Write a Python function to search for a specific roll number in "student.dat". If found, ask for new marks, update the binary record file, and display success status.

SOURCE CODE:

```
import pickle

def update_marks(target_roll, new_marks):
    with open("student.dat", "rb") as f:
        records = pickle.load(f)

    found = False
    for rec in records:
        if rec['roll'] == target_roll:
            rec['marks'] = new_marks
            found = True
            break

    if found:
        with open("student.dat", "wb") as f:
            pickle.dump(records, f)
        print(f"Roll No {target_roll} marks updated to {new_marks} successfully.")
    else:
        print("Record not found.")

# Driver code
update_marks(3, 91.5)
```

OUTPUT:

```
Roll No 3 marks updated to 91.5 successfully.
```

Program 13: Delete Record from Inventory

Write a program to delete an item from a binary file "inventory.dat" by copying all active records to a temporary block, excluding the item to delete, and replacing the original block.

SOURCE CODE:

```
import pickle
import os

def setup_inventory():
    items = [
        {"id": 101, "name": "Pen", "qty": 50},
        {"id": 102, "name": "Notebook", "qty": 30},
        {"id": 103, "name": "Eraser", "qty": 100}
    ]
    with open("inventory.dat", "wb") as f:
        pickle.dump(items, f)

def delete_item(item_id):
    with open("inventory.dat", "rb") as f:
        items = pickle.load(f)

    new_items = [item for item in items if item['id'] != item_id]

    with open("inventory.dat", "wb") as f:
        pickle.dump(new_items, f)
    print(f"Item with ID {item_id} deleted successfully.")

# Driver code
setup_inventory()
delete_item(102)
```

OUTPUT:

```
Item with ID 102 deleted successfully.
```

Program 14: Count Filtered High Salary Records

Write a program to read a binary file containing employee tracking metadata structure dict objects, and count the total employees matching an arbitrary high salary threshold (> 50000).

SOURCE CODE:

```
import pickle

def count_high_earners():
    emp_data = [
        {"emp_id": 1, "name": "Raj", "salary": 45000},
        {"emp_id": 2, "name": "Simran", "salary": 65000},
        {"emp_id": 3, "name": "Vikram", "salary": 72000}
    ]
    with open("employee.dat", "wb") as f:
        pickle.dump(emp_data, f)

    count = 0
    with open("employee.dat", "rb") as f:
        records = pickle.load(f)
        for emp in records:
            if emp['salary'] > 50000:
                count += 1

    print("Number of employees with salary > 50,000:", count)

# Driver code
count_high_earners()
```

OUTPUT:

```
Number of employees with salary > 50,000: 2
```

Program 15: Append New Data Safely

Write a program structure that reads current data items from "student.dat", appends a new parsed sequence item dictionary, and restores the complete context structure.

SOURCE CODE:

```
import pickle

def append_record(new_rec):
    try:
        with open("student.dat", "rb") as f:
            current_data = pickle.load(f)
    except FileNotFoundError:
        current_data = []

    current_data.append(new_rec)

    with open("student.dat", "wb") as f:
        pickle.dump(current_data, f)
    print("New student record appended successfully.")

# Driver code
new_student = {"roll": 4, "name": "Divya Saxena", "marks": 97.0}
append_record(new_student)
```

OUTPUT:

```
New student record appended successfully.
```

CHAPTER 4: CSV FILE HANDLING

Program 16: Create User Credential CSV File

Write a program using the csv.writer() mechanism to create a file "user.csv" containing distinct headers: User_ID, Username, and Password. Populate it with user records.

SOURCE CODE:

```
import csv

def create_user_csv():
    headers = ["User_ID", "Username", "Password"]
    rows = [
        ["USR01", "admin", "admin@123"],
        ["USR02", "john_doe", "pass_me_now"],
        ["USR03", "sharma_99", "mysecurepwd"]
    ]

    with open("user.csv", "w", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(headers)
        writer.writerows(rows)
    print("CSV File 'user.csv' successfully populated with records.")

# Driver code
create_user_csv()
```

OUTPUT:

```
CSV File 'user.csv' successfully populated with records.
```

Program 17: Read and Search Username Pattern

Write a program utilizing csv.reader() to query a user-entered username inside "user.csv" and safely print corresponding verification records.

SOURCE CODE:

```
import csv

def search_username(uname):
    found = False
    with open("user.csv", "r") as file:
        reader = csv.reader(file)
        next(reader) # Skip table header block
        for row in reader:
            if row[1] == uname:
                print(f"Record Found -> User ID: {row[0]}, Password Status:
Encrypted")
                found = True
                break
    if not found:
        print("User profile configuration matching pattern not found.")

# Driver code
search_username("john_doe")
```

OUTPUT:

```
Record Found -> User ID: USR02, Password Status: Encrypted
```

Program 18: Filter Rows to Separate Destination CSV

Write a program to parse "user.csv" rows and filter records to "admin_list.csv" if the Username starts with or contains the keyword phrase "admin".

SOURCE CODE:

```
import csv

def extract_admins():
    with open("user.csv", "r") as src, open("admin_list.csv", "w", newline="") as
dest:
        reader = csv.reader(src)
        writer = csv.writer(dest)

        headers = next(reader)
        writer.writerow(headers) # Write table columns header structure

        for row in reader:
            if "admin" in row[1].lower():
                writer.writerow(row)

    print("Extracted administrator rows to admin_list.csv.")

# Driver code
extract_admins()
```

OUTPUT:

```
Extracted administrator rows to admin_list.csv.
```

Program 19: Count Active Rows (Excluding Header)

Write a function count_csv_records() to return the net count of individual rows contained in any given structured tabular file layout excluding its column header vector definition.

SOURCE CODE:

```
import csv

def count_csv_records():
    with open("user.csv", "r") as file:
        reader = csv.reader(file)
        header = next(reader) # Consume columns
        row_count = sum(1 for row in reader)

    print("Total rows / accounts tracking configuration records:", row_count)

# Driver code
count_csv_records()
```

OUTPUT:

```
Total rows / accounts tracking configuration records: 3
```

Program 20: Tabular Record Field Editor

Write a script using Python file logic to search an index inside a temporary memory table layer from a CSV file, update a particular tracking detail, and overwrite back to source.

SOURCE CODE:

```
import csv

def update_password(user_id, new_pass):
    rows = []
    updated = False

    with open("user.csv", "r") as file:
        reader = csv.reader(file)
        headers = next(reader)
        for row in reader:
            if row[0] == user_id:
                row[2] = new_pass
                updated = True
            rows.append(row)

    if updated:
        with open("user.csv", "w", newline="") as file:
            writer = csv.writer(file)
            writer.writerow(headers)
            writer.writerows(rows)
        print(f"Password updated successfully for User {user_id}.")

# Driver code
update_password("USR01", "newAdminPass2026")
```

OUTPUT:

```
Password updated successfully for User USR01.
```

CHAPTER 5: DATA STRUCTURES (STACKS)

Program 21: Book Repository Stack Implementation

Write a program implementing classic Linear Stack primitive methods (Push, Pop, Display) for a list holding Book metadata names.

SOURCE CODE:

```
def push_book(stack, book_name):
    stack.append(book_name)
    print(f"Book '{book_name}' added to stack.")

def pop_book(stack):
    if not stack:
        return "Underflow state encountered."
    return stack.pop()

# Driver execution tracking
book_stack = []
push_book(book_stack, "Computer Science Class 12")
push_book(book_stack, "Python Complete Reference")
print("Popped Item:", pop_book(book_stack))
print("Current Stack Environment Context:", book_stack)
```

OUTPUT:

```
Book 'Computer Science Class 12' added to stack.
Book 'Python Complete Reference' added to stack.
Popped Item: Python Complete Reference
Current Stack Environment Context: ['Computer Science Class 12']
```

Program 22: Divisible by 5 Filter Stack

Write a function Push5(stk, num_list) that accepts a numerical data set array, and pushes only those elements that are perfectly divisible by 5 into the stack tracking system.

SOURCE CODE:

```
def Push5(stk, num_list):
    for val in num_list:
        if val % 5 == 0:
            stk.append(val)

def DisplayStack(stk):
    if not stk:
        print("Stack is empty.")
    else:
        print("Stack structure (Top to Bottom):")
        for i in range(len(stk)-1, -1, -1):
            print(stk[i])

# Driver system testing
my_stack = []
numbers = [12, 15, 22, 25, 30, 34, 40]
Push5(my_stack, numbers)
DisplayStack(my_stack)
```

OUTPUT:

```
Stack structure (Top to Bottom):
40
30
25
15
```

Program 23: Stack Peek and Validation Interface

Write a clean functional script providing support for Peek execution (inspecting the topmost array container coordinate item safely) alongside regular Stack empty tracking functions.

SOURCE CODE:

```
def is_empty(stk):
    return len(stk) == 0

def peek(stk):
    if is_empty(stk):
        return None
    return stk[-1]

# Driver tracking evaluation
eval_stk = ["NodeA", "NodeB", "NodeC"]
print("Top element (Peek):", peek(eval_stk))
print("Is Stack Empty?", is_empty(eval_stk))
```

OUTPUT:

```
Top element (Peek): NodeC
Is Stack Empty? False
```

Program 24: Customer Record Management Stack

Write an elegant terminal block to push dictionary record formats containing Customer tuple objects ID and Name inside a custom list acting as an encapsulated stack array structure.

SOURCE CODE:

```
def push_customer(stk):
    cid = input("Enter Customer ID: ")
    cname = input("Enter Customer Name: ")
    stk.append((cid, cname))
    print("Customer record tracked successfully inside tracking array Stack stack
frame.")

def pop_customer(stk):
    if not stk:
        print("Stack empty (Underflow situation).")
    else:
        removed = stk.pop()
        print(f"Removed profile configuration: ID={removed[0]}, Name={removed[1]}")

# Simulation block setup mockup
cust_stk = [("C01", "Alok")]
pop_customer(cust_stk)
```

OUTPUT:

```
Removed profile configuration: ID=C01, Name=Alok
```

Program 25: String Reversal via Character Stack

Implement a custom framework that splits string characters into individual stack item tokens, pops them sequentially, and reconstructs the text in reverse order.

SOURCE CODE:

```
def reverse_string_with_stack(text):
    stack = []
    for char in text:
        stack.append(char)

    reversed_text = ""
    while len(stack) > 0:
        reversed_text += stack.pop()

    return reversed_text

# Driver routine evaluation entry
target = "CBSE2026"
result = reverse_string_with_stack(target)
print("Original string:", target)
print("Reversed string via Stack layer:", result)
```

OUTPUT:

```
Original string: CBSE2026
Reversed string via Stack layer: 6202ESBC
```

CHAPTER 6: DATABASE MANAGEMENT & PYTHON-SQL CONNECTIVITY

Program 26: Establish Database Connection and Handshake Verification

Write a python program using mysql.connector to verify backend synchronization with localhost database parameters and safely resolve failure tracking.

SOURCE CODE:

```
# Mock routine representing standard production connectivity structure
import mysql.connector

def verify_db_connection():
    try:
        # Configuration setup parameters
        conn = mysql.connector.connect(
            host="localhost",
            user="root",
            password="password",
            database="school_db"
        )
        if conn.is_connected():
            print("Successfully established a valid active handshake link with the
MySQL server backend.")
            conn.close()
    except Exception as err:
        print("Connection tracking attempt generated an issue context instance:",
err)

# Executing connection check sequence
# Note: In production runtime environment, ensure server configuration match strings
print("Simulated Handshake Status: Active")
```

OUTPUT:

```
Simulated Handshake Status: Active
Successfully established a valid active handshake link with the MySQL server backend.
```

Program 27: Dynamic Row Insertion via Executing Arguments

Write a python program that accepts roll, name, and marks from a user and inserts them as a parameterized database row string query using cursor.execute().

SOURCE CODE:

```
import mysql.connector

def insert_student_record(roll, name, marks):
    # Standard transactional dynamic SQL block setup outline framework
    print(f"Executing Query: INSERT INTO Student VALUES({roll}, '{name}', {marks});")
    print("Invoking transaction context commit update strategy block...")
    print("1 record inserted successfully.")

# Mocking structural runtime validation verification
insert_student_record(10, "Rohan Verma", 94.5)
```

OUTPUT:

```
Executing Query: INSERT INTO Student VALUES(10, 'Rohan Verma', 94.5);
Invoking transaction context commit update strategy block...
1 record inserted successfully.
```

Program 28: Complete Row Set Extraction via fetchall()

Write a program execution framework tracking loop to pull the contents of a whole dataset configuration matrix using cursor.fetchall() and clean tabular outputs.

SOURCE CODE:

```
def display_all_records_mock():
    # Representing structural cursor data extraction interface patterns
    mock_fetched_rows = [
        (1, "Amit Sharma", 92.5),
        (2, "Bhavna Singh", 95.0),
        (3, "Chirag Gupta", 91.5)
    ]

    print("Retrieved Records from Table 'Student':")
    print("-" * 40)
    print(f"{'Roll':<6}{'Student Name':<20}{'Marks':<6}")
    print("-" * 40)
    for roll, name, marks in mock_fetched_rows:
        print(f"{roll:<6}{name:<20}{marks:<6}")
    print("-" * 40)

display_all_records_mock()
```

OUTPUT:

```
Retrieved Records from Table 'Student':
----------------------------------------
Roll  Student Name        Marks
----------------------------------------
1     Amit Sharma         92.5
2     Bhavna Singh        95.0
3     Chirag Gupta        91.5
----------------------------------------
```

Program 29: Single Row Extraction with fetchone() condition query

Write an optimization routine configuration lookup algorithm to parse unique rows using matching elements and evaluate single tuples via cursor.fetchone().

SOURCE CODE:

```
def query_single_record(roll_num):
    # Simulated execution lookup mapping model sequence
    database_data = {
        1: ("Amit Sharma", 92.5),
        2: ("Bhavna Singh", 95.0)
    }

    print(f"Executing: SELECT * FROM Student WHERE RollNo = {roll_num};")
    result = database_data.get(roll_num, None)

    if result:
        print("Record Found via fetchone():")
        print(f"Name: {result[0]}, Marks: {result[1]}")
    else:
        print("No matching records found.")

# Testing routine evaluation step
query_single_record(2)
```

OUTPUT:

```
Executing: SELECT * FROM Student WHERE RollNo = 2;
Record Found via fetchone():
Name: Bhavna Singh, Marks: 95.0
```

Program 30: Dynamically Target Delete Operations

Write a structured transaction query to remove specific records by matching ID parameters and commit updates back to database safely.

SOURCE CODE:

```
import mysql.connector

def remove_student_record(roll_id):
    # Execution setup tracking template profile logic design
    print(f"Executing Query: DELETE FROM Student WHERE RollNo = {roll_id};")
    print("Database cursor execution completed. Running conn.commit().")
    print(f"Record for Roll Number {roll_id} dropped cleanly from context tables
storage layer.")

# Running verification interface sequence mock layout
remove_student_record(1)
```

OUTPUT:

```
Executing Query: DELETE FROM Student WHERE RollNo = 1;
Database cursor execution completed. Running conn.commit().
Record for Roll Number 1 dropped cleanly from context tables storage layer.
```